

Toward OS-Portable Hypervisor Cores

Abstract

Modern hypervisors are increasingly used outside traditional server virtualization, including embedded systems, safety-critical platforms, edge devices, and cloud-native isolation. Despite this diversity, most hypervisors remain tightly coupled to a particular substrate environment. KVM is deeply integrated with Linux, static-partitioning hypervisors such as Bao and Jailhouse assume specific boot, memory, and device-management models, and recent safety-critical hypervisor studies largely focus on hardware partitioning, temporal isolation, or guest support rather than the portability of the hypervisor itself across different host operating systems [4, 7, 5, 6]. This coupling makes it difficult for emerging operating systems, unikernels, and research kernels to reuse mature virtualization logic without reimplementing a hypervisor or deeply restructuring their kernels.

This paper presents an OS-portable hypervisor architecture that decouples hypervisor logic from the substrate operating system. Starting from AxVisor, a hypervisor originally built on ArceOS, we separate the virtualization core from the OS services on which it depends and define a narrow substrate interface for memory management, CPU and vCPU control, interrupt and timer handling, synchronization, device access, and lifecycle management. We then retarget the same AxVisor core to three substantially different substrate environments: ArceOS, Asterinas, and Linux. This design turns the substrate OS from an implicit implementation dependency into an explicit service provider, allowing one hypervisor core to be reused across different kernel architectures.

Our experience shows that substrate-OS portability is both feasible and non-trivial. The main challenge is not syntactic API wrapping, but reconciling semantic differences in physical memory ownership, address translation, interrupt context, scheduling, device access, and initialization order across operating systems. By quantifying the size of the substrate interface, the amount of reused hypervisor code, and the runtime overheads across three backends, we demonstrate that a hypervisor can be engineered as a portable virtualization component rather than as a subsystem bound to a single OS. The resulting architecture provides a practical path for bringing virtualization support to emerging operating systems while preserving the investment in an existing hypervisor implementation.

1 Introduction

Virtualization has become a foundational mechanism for isolation, consolidation, and system construction. In cloud platforms, lightweight virtual machine monitors support serverless workloads and container-strength deployment models with stronger isolation than shared-kernel containers [3, 2]. In embedded and safety-critical systems, hypervisors are used to consolidate heterogeneous workloads with different timing and assurance requirements on a single multicore platform [5, 1]. In industrial, automotive, and edge scenarios, it is increasingly common to run a feature-rich general-purpose OS alongside real-time operating systems, bare-metal components, or safety-certified software stacks. Across these domains, the hypervisor is no longer merely a server consolidation layer; it is a core system component for constructing heterogeneous software platforms.

Existing hypervisor designs primarily address two forms of portability. The first is guest portability: a hypervisor should support different guest operating systems or runtime environments. The second is hardware portability: a hypervisor should run on different instruction-set architectures or platform configurations, such as x86, Arm, or RISC-V [9, 10]. These dimensions have received substantial attention in both general-purpose and embedded virtualization. Xen, KVM, ACRN, Bao, Jailhouse, and seL4-based VMMs all provide different answers to the question of how to expose hardware resources to guests, how to isolate virtual machines, and how to trade generality against predictability [4, 7, 6]. However, a third form of portability remains underexplored: whether the hypervisor itself can be reused across different substrate operating systems.

This missing dimension matters because modern systems increasingly require virtualization support in environments that are not traditional hypervisor hosts. A new kernel may need to run virtual machines for compatibility, testing, isolation, or cloud-native execution. A unikernel or library OS may need a compact virtualization layer without importing a full general-purpose kernel. A Rust-based experimental kernel may want to reuse existing hypervisor

logic while enforcing a different safety and memory model. Conversely, a mature OS such as Linux may provide rich drivers, scheduling, and management facilities, but its internal abstractions differ significantly from those of a unikernel or a microkernel-like system. If each substrate OS must build its own hypervisor from scratch, virtualization support becomes expensive, fragmented, and difficult to validate.

The central observation of this paper is that a hypervisor depends on a substrate OS through a relatively small but semantically rich set of services. These services include allocating and mapping memory, creating and controlling execution contexts, routing interrupts, managing timers, synchronizing concurrent control paths, accessing devices, and coordinating initialization and teardown. Much of the hypervisor’s core logic, such as VM construction, vCPU state management, trap handling, guest memory bookkeeping, and virtual device mediation, need not be tied to a particular OS implementation. What prevents reuse is not the absence of a hardware abstraction layer, but the lack of a precise contract between the hypervisor and the OS that hosts it.

We call this contract the substrate interface. Unlike a conventional hardware abstraction layer, the substrate interface abstracts operating-system services rather than device registers or processor features. It must expose enough low-level control for virtualization while hiding the accidental details of each substrate OS. This makes its design delicate. If the interface is too narrow, the hypervisor cannot implement necessary functions such as guest memory mapping, interrupt injection, or device passthrough. If it is too close to a particular OS, portability is lost. If it ignores semantic differences, the same hypervisor code may appear portable while relying on assumptions that only hold on one substrate. A useful interface must therefore be small, explicit, and faithful to the semantics required by virtualization.

This paper explores substrate-OS portability through AxVisor. AxVisor was originally built on ArceOS, whose unikernel-style design gives the hypervisor direct and lightweight access to system services. We restructure AxVisor into two parts: a portable hypervisor core and a substrate-specific adaptation layer. The core implements the reusable virtualization logic, while each adaptation layer implements the substrate interface for a particular OS. We then implement three backends for ArceOS, Asterinas, and Linux. These substrates represent different points in the OS design space: ArceOS provides a lightweight modular kernel base, Asterinas represents a different kernel architecture and safety model, and Linux provides a mature monolithic kernel with extensive device and management facilities. Supporting all three allows us to evaluate whether the abstraction captures the common requirements of hypervisor construction rather than the idiosyncrasies of a single system.

The resulting system changes the role of the substrate OS. Instead of being the environment in which the hypervisor is manually embedded, the substrate OS becomes a provider of explicitly specified hypervisor-grade services. This shift enables the same virtualization core to follow different deployment requirements. A lightweight substrate can be used when a small trusted computing base and short control paths are desirable. A full-featured substrate can be used when driver support, debugging tools, and mature scheduling facilities are more important. A research kernel can obtain virtualization support by implementing the substrate interface rather than by reimplementing a hypervisor. In this sense, OS-portable hypervisors complement existing work on guest portability, hardware portability, and static partitioning [7, 6, 8].

Our work makes three contributions. First, we identify substrate-OS portability as a distinct design goal for hypervisors and analyze why existing hypervisor architectures are difficult to retarget across OS boundaries. Second, we design and implement a substrate-agnostic AxVisor architecture that separates reusable virtualization logic from OS-specific services through a narrow substrate interface. Third, we realize the design on ArceOS, Asterinas, and Linux, and use this experience to characterize the engineering effort, semantic mismatches, and performance costs of building a hypervisor that can run on multiple substrate operating systems. The broader lesson is that hypervisors need not be inseparable from a single host OS; with the right service boundary, they can be engineered as reusable system components.

References

- [1] Alessandro Cilardo. Virtualization for embedded and cyber-physical systems: A survey. *ACM Computing Surveys*, 2022.
- [2] Tom Goethals, Bruno Volckaert, and Filip De Turck. A functional and performance evaluation of lightweight virtualization platforms for serverless computing. In *Proceedings of the 2022 IEEE International Conference on Cloud Computing Technology and Science*, 2022.

- [3] Connor Holmes, Laszlo Szekeres, and Dan R. K. Ports. Severifast: Lightweight virtualization for fast serverless computing. *arXiv preprint*, 2024.
- [4] Eddie Dong Li, Yaozu Xia, Baosheng Zang, and Haibing Guan. ACRN: A big little hypervisor for IoT development. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, pages 31–44. ACM, 2019.
- [5] Soledad Lozano, Benjamin Lesage, Laurent George, and Mohamed Abid. A comprehensive survey on hypervisors for embedded systems. *ACM Computing Surveys*, 2023.
- [6] José Martins, Adriano Tavares, and Sandro Pinto. Shedding light on static partitioning hypervisors for arm-based mixed-criticality systems. In *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium*, 2023.
- [7] José Martins, Adriano Tavares, Marco Solieri, Marko Bertogna, and Sandro Pinto. Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems. In *Workshop on Next Generation Real-Time Embedded Systems*, 2020.
- [8] Tiago Peixoto, José Martins, and Sandro Pinto. Birtio: Virtio for bare-metal and real-time systems. In *Proceedings of the 2024 IEEE Real-Time Systems Symposium Workshops*, 2024.
- [9] Bruno Sousa, José Martins, and Sandro Pinto. A first look at the risc-v virtualization extension. In *Proceedings of the 18th ACM International Conference on Computing Frontiers*, 2021.
- [10] Bruno Sousa, José Martins, and Sandro Pinto. HSPV: Hardware support for processor virtualization. *ACM Computing Surveys*, 2024.